VRUSHABH SAWANT SYVLSI_44 125BTVL1033

Experiment 7

Academic Year 2026-2027			
Program: Electronics Engineering (VLSI Design and Technology)			
Year of Study	Second Year	**Semester**	III
Course	Skill Lab: Python Programming	**Course Code**	VDCOR2VS201
Class	SY VLSI	**Course In-Charge**	Mr. Nirmol Munvar
Exp Name	Implement Class, Data class variables, instance variables, self keyword, constructor		
CO	Apply object-oriented programming concepts such as classes, inheritance, and polymorphism to develop modular Python applications.		

Classes in Python

A **class** is a blueprint for creating objects. It defines the data and behavior that objects created from the class can have.
A class is defined using the class keyword.

```
class Student:
    pass
```

An object is created from a class by calling the class name.

```
student1 = Student()
```

Here, Student is the class and student1 is an object of that class.

Constructor

A **constructor** is a special method that is automatically executed when an object is created.
In Python, the constructor is generally implemented using the __init__() method.

```
class Student:
    def __init__(self, name, branch):
        self.name = name
        self.branch = branch

student1 = Student("Rahul", "VLSI")

print(student1.name)
print(student1.branch)
```

When Student("Rahul", "VLSI") is executed, Python automatically calls __init__().
The constructor is commonly used to initialize the attributes of an object.

self Keyword

self refers to the **current object** of the class.
It is used to access the instance variables and instance methods belonging to that particular object.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

student1 = Student("Rahul", 85)
```

```
student2 = Student("Priya", 92)

print(student1.name)
print(student2.name)
```
Here, self.name refers to the name belonging to the current object.
For student1, self refers to student1.
For student2, self refers to student2.
The self parameter is explicitly written in the method definition, but it is automatically supplied when the method is called through an object.
Instance Variables
Instance variables are variables that belong to a particular object. Each object can have its own separate values for these variables.
They are generally created using self.
```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

student1 = Student("Rahul", 80)
student2 = Student("Priya", 90)

print(student1.name)
print(student2.name)
```
Here:
self.name
self.marks
are instance variables.
student1 and student2 have separate copies of these variables.
Changing one object's instance variable does not normally change the other object's instance variable.
```
student1.marks = 85

print(student1.marks)
print(student2.marks)
```
The output will be:
85
90
Class Variables
A **class variable** belongs to the class rather than to an individual object. It is shared by all objects of that class.
It is normally declared directly inside the class but outside its methods.
```
class Student:
    college = "ABC Institute"

    def __init__(self, name):
        self.name = name

student1 = Student("Rahul")
student2 = Student("Priya")
```

```
print(student1.college)
print(student2.college)
```
Both objects access the same class variable college.
A class variable is useful when a value is common to all objects.
For example:
```
class Student:
    college = "ABC Institute"
```
The college name can be common to every student, while the student's name is different for each object.

Class Variables vs Instance Variables

Feature	**Class Variable**	**Instance Variable**
Belongs to	Class	Individual object
Shared among objects	Yes	No
Usually declared	Directly inside class	Using self
Example	college	name, marks
Separate value for each object	No	Yes

Example combining both:
```
class Student:
    college = "ABC Institute"

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

student1 = Student("Rahul", 85)
student2 = Student("Priya", 92)

print(student1.name)
print(student1.marks)
print(student1.college)

print(student2.name)
print(student2.marks)
print(student2.college)
```
Here, college is a **class variable**, while name and marks are **instance variables**.

Basic Exercises

Exercise 1: Student Class

Create a Student class with a constructor that accepts the student's name, roll number, and marks.
Use instance variables to store these values.
Create two student objects and display their details.

Exercise 2: Class and Instance Variables

Create an Employee class with:
- A class variable company = "ABC Technologies"
- Instance variables name, employee_id, and salary

Create three employee objects and display their details.
Change the company name using the class variable and observe its effect on all objects.

Exercise 3: Using self

Create a Rectangle class with instance variables length and width.
Create methods using self to calculate:

- Area
- Perimeter

Create two rectangle objects with different dimensions and display their areas and perimeters.

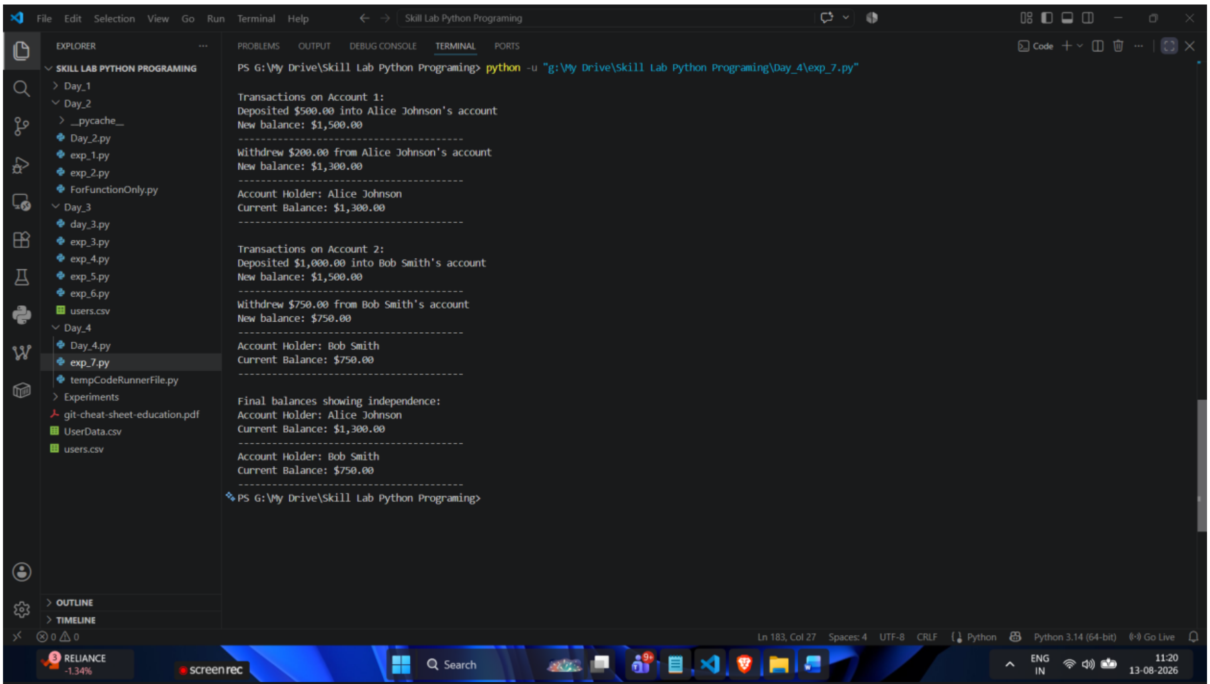
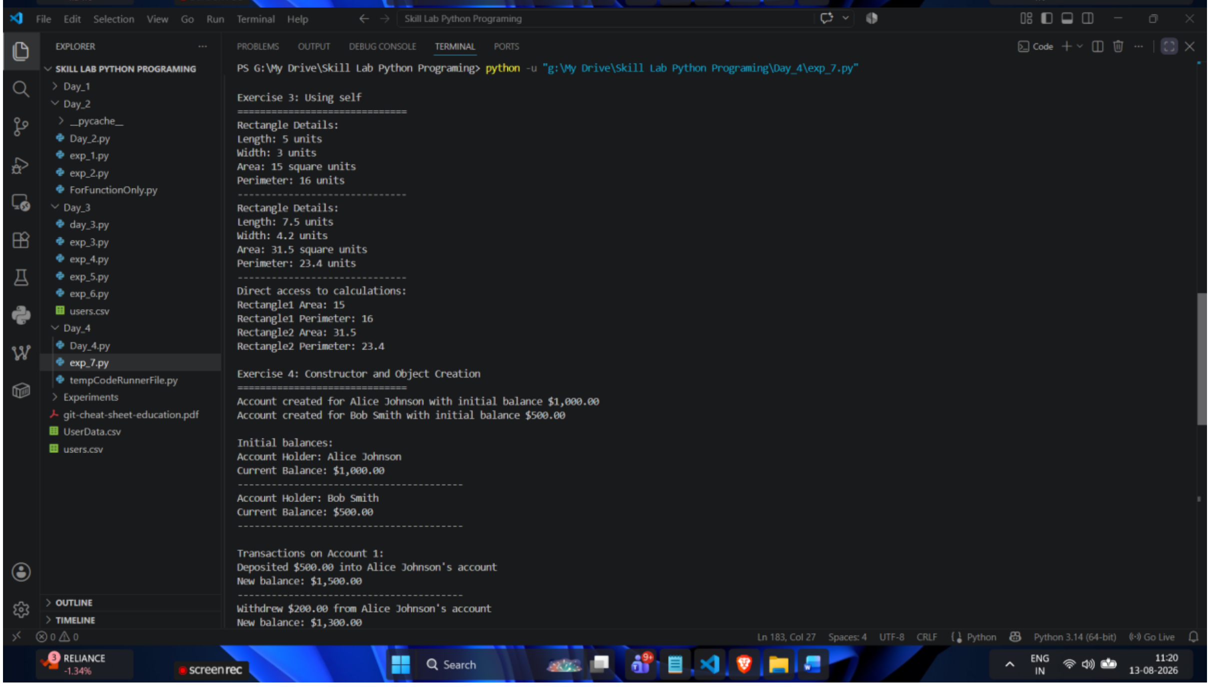
Exercise 4: Constructor and Object Creation

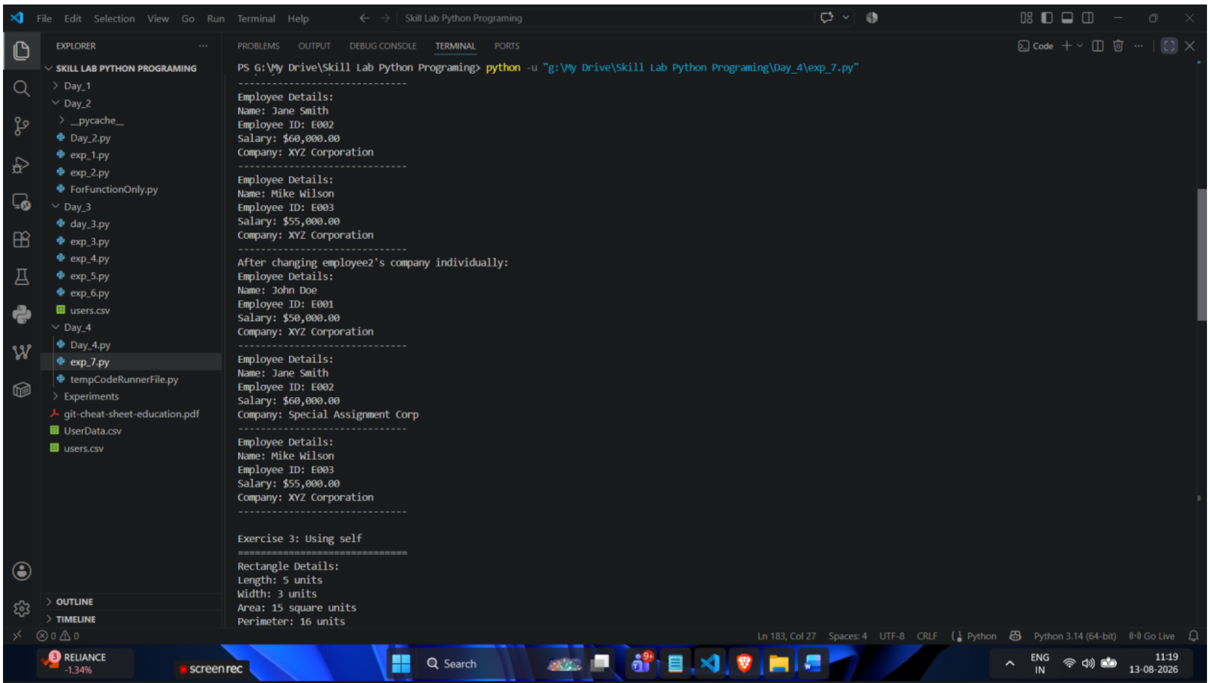
Create a BankAccount class with a constructor that accepts the account holder's name and initial balance.

Create methods to:

- Deposit money
- Withdraw money
- Display the current balance

Use self to access and modify the instance variables. Create two bank account objects and demonstrate that their balances are maintained independently.


PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_7.py"
Transactions on Account 1:
Deposited $500.00 into Alice Johnson's account
New balance: $1,500.00
Withdrew $200.00 from Alice Johnson's account
New balance: $1,300.00
Account Holder: Alice Johnson
Current Balance: $1,300.00
Transactions on Account 2:
Deposited $1,000.00 into Bob Smith's account
New balance: $1,500.00
Withdrew $750.00 from Bob Smith's account
New balance: $750.00
Account Holder: Bob Smith
Current Balance: $750.00
Final balances showing independence:
Account Holder: Alice Johnson
Current Balance: $1,300.00
Account Holder: Bob Smith
Current Balance: $750.00
PS G:\My Drive\Skill Lab Python Programing>

PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_7.py"
Exercise 3: Using self
Rectangle Details:
Length: 5 units
Width: 3 units
Area: 15 square units
Perimeter: 16 units
Rectangle Details:
Length: 7.5 units
Width: 4.2 units
Area: 31.5 square units
Perimeter: 23.4 units
Direct access to calculations:
Rectangle1 Area: 15
Rectangle1 Perimeter: 16
Rectangle2 Area: 31.5
Rectangle2 Perimeter: 23.4
Exercise 4: Constructor and Object Creation
Account created for Alice Johnson with initial balance $1,000.00
Account created for Bob Smith with initial balance $500.00
Initial balances:
Account Holder: Alice Johnson
Current Balance: $1,000.00
Account Holder: Bob Smith
Current Balance: $500.00
Transactions on Account 1:
Deposited $500.00 into Alice Johnson's account
New balance: $1,500.00
Withdrew $200.00 from Alice Johnson's account
New balance: $1,300.00


PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_7.py"

Employee Details:
Name: Jane Smith
Employee ID: E002
Salary: $60,000.00
Company: XYZ Corporation

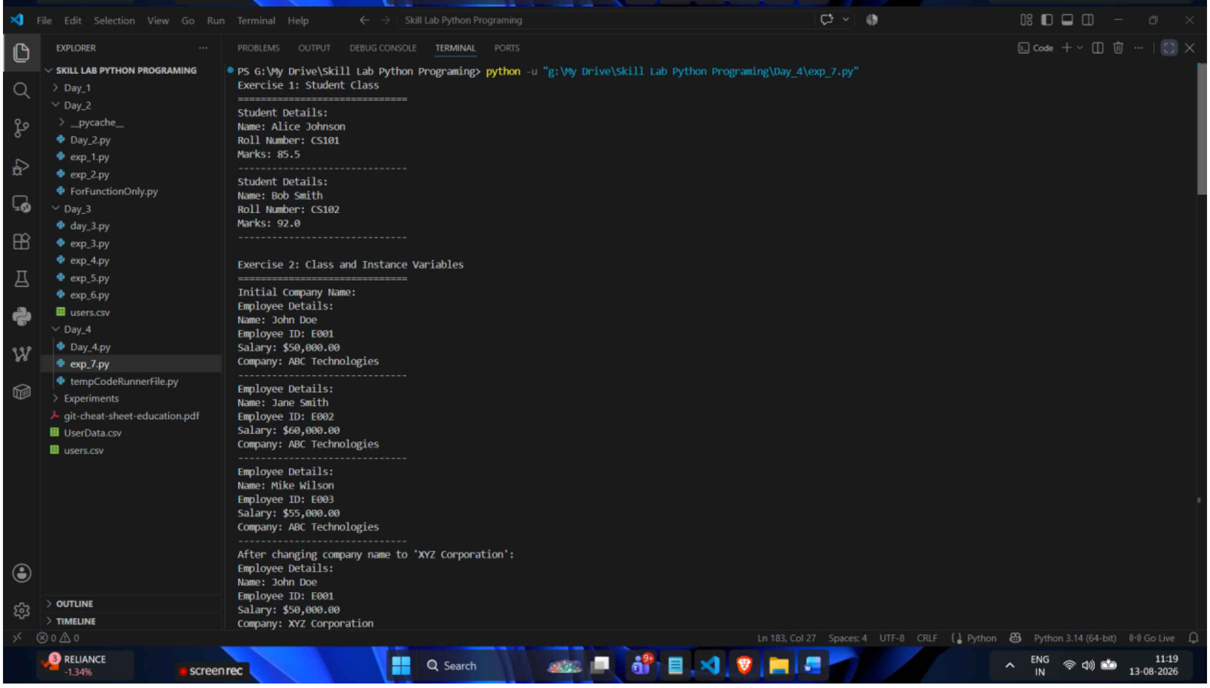
Employee Details:
Name: Mike Wilson
Employee ID: E003
Salary: $55,000.00
Company: XYZ Corporation

After changing employee2's company individually:
Employee Details:
Name: John Doe
Employee ID: E001
Salary: $50,000.00
Company: XYZ Corporation

Employee Details:
Name: Jane Smith
Employee ID: E002
Salary: $60,000.00
Company: Special Assignment Corp

Employee Details:
Name: Mike Wilson
Employee ID: E003
Salary: $55,000.00
Company: XYZ Corporation

Exercise 3: Using self
==============================
Rectangle Details:
Length: 5 units
Width: 3 units
Area: 15 square units
Perimeter: 16 units


PS G:\My Drive\Skill Lab Python Programing> python -u "g:\My Drive\Skill Lab Python Programing\Day_4\exp_7.py"
Exercise 1: Student Class
==============================
Student Details:
Name: Alice Johnson
Roll Number: CS101
Marks: 85.5

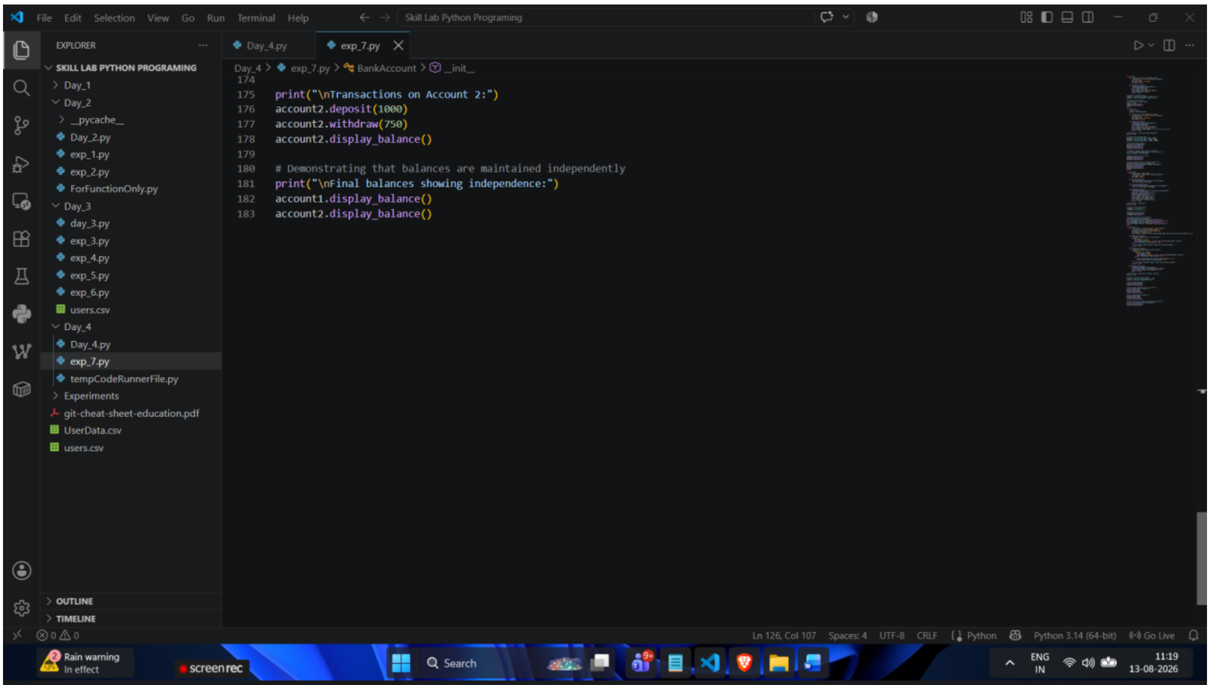
Student Details:
Name: Bob Smith
Roll Number: CS102
Marks: 92.0

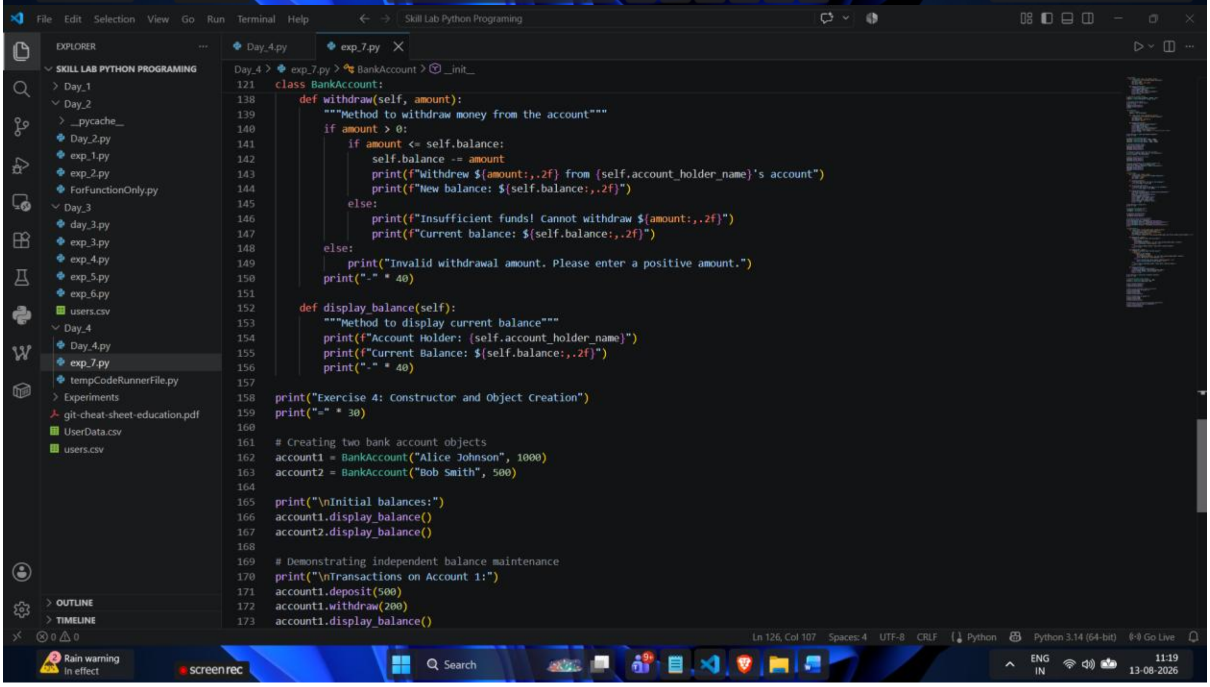
Exercise 2: Class and Instance Variables
==============================
Initial Company Name:
Employee Details:
Name: John Doe
Employee ID: E001
Salary: $50,000.00
Company: ABC Technologies

Employee Details:
Name: Jane Smith
Employee ID: E002
Salary: $60,000.00
Company: ABC Technologies

Employee Details:
Name: Mike Wilson
Employee ID: E003
Salary: $55,000.00
Company: ABC Technologies

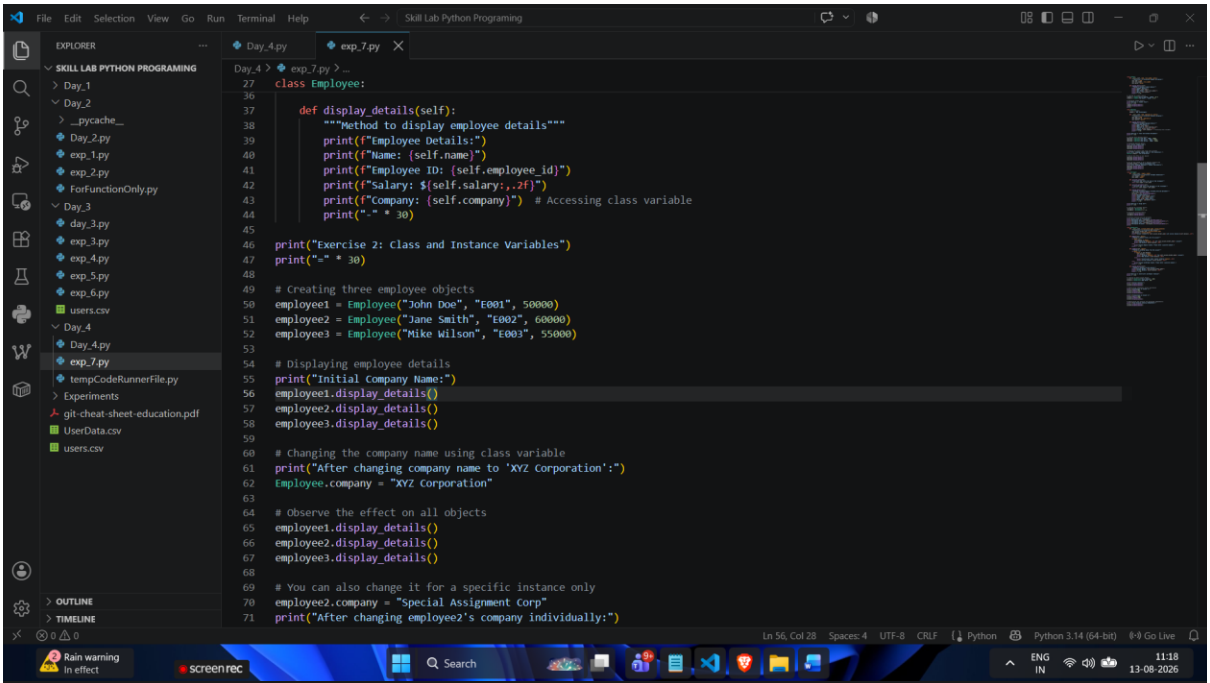
After changing company name to 'XYZ Corporation':
Employee Details:
Name: John Doe
Employee ID: E001
Salary: $50,000.00
Company: XYZ Corporation


Skill Lab Python Programing
Day_4.py
exp_7.py
Day_4 > exp_7.py > BankAccount > __init__
print("\nTransactions on Account 2:")
account2.deposit(1000)
account2.withdraw(750)
account2.display_balance()
# Demonstrating that balances are maintained independently
print("\nFinal balances showing independence:")
account1.display_balance()
account2.display_balance()

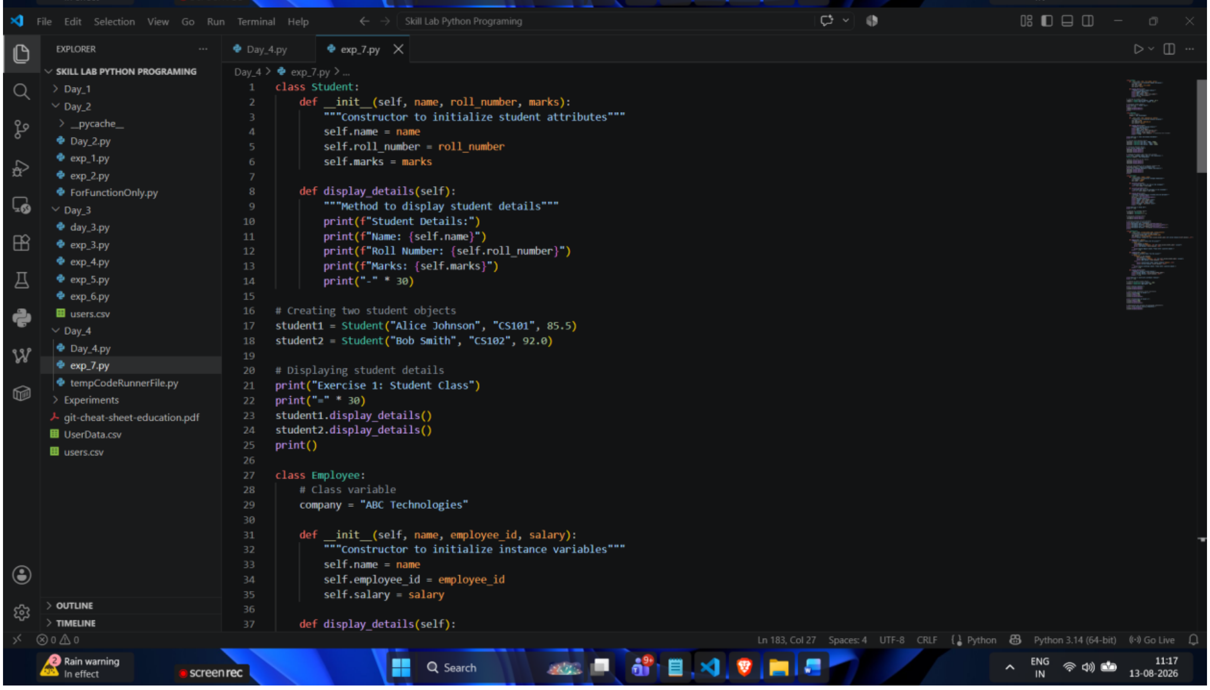

Skill Lab Python Programing
Day_4.py
exp_7.py
Day_4 > exp_7.py > BankAccount > __init__
class BankAccount:
    def withdraw(self, amount):
        """Method to withdraw money from the account"""
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
                print(f"Withdrew ${amount:,.2f} from {self.account_holder_name}'s account")
                print(f"New balance: ${self.balance:,.2f}")
            else:
                print(f"Insufficient funds! Cannot withdraw ${amount:,.2f}")
                print(f"Current balance: ${self.balance:,.2f}")
        else:
            print("Invalid withdrawal amount. Please enter a positive amount.")
        print("-" * 40)
    def display_balance(self):
        """Method to display current balance"""
        print(f"Account Holder: {self.account_holder_name}")
        print(f"Current Balance: ${self.balance:,.2f}")
        print("-" * 40)
print("Exercise 4: Constructor and Object Creation")
print("=" * 30)
# Creating two bank account objects
account1 = BankAccount("Alice Johnson", 1000)
account2 = BankAccount("Bob Smith", 500)
print("\nInitial balances:")
account1.display_balance()
account2.display_balance()
# Demonstrating independent balance maintenance
print("\nTransactions on Account 1:")
account1.deposit(500)
account1.withdraw(200)
account1.display_balance()

```python
# Creating two rectangle objects
rectangle1 = Rectangle(5, 3)
rectangle2 = Rectangle(7.5, 4.2)

# Displaying rectangle details
rectangle1.display_details()
rectangle2.display_details()

# Directly accessing area and perimeter
print("Direct access to calculations:")
print(f"Rectangle1 Area: {rectangle1.calculate_area()}")
print(f"Rectangle1 Perimeter: {rectangle1.calculate_perimeter()}")
print(f"Rectangle2 Area: {rectangle2.calculate_area()}")
print(f"Rectangle2 Perimeter: {rectangle2.calculate_perimeter()}")
print()

class BankAccount:
    def __init__(self, account_holder_name, initial_balance):
        """Constructor to initialize bank account"""
        self.account_holder_name = account_holder_name
        self.balance = initial_balance
        print(f"Account created for {self.account_holder_name} with initial balance ${self.balance:,.2f}")

    def deposit(self, amount):
        """Method to deposit money into the account"""
        if amount > 0:
            self.balance += amount
            print(f"Deposited ${amount:,.2f} into {self.account_holder_name}'s account")
            print(f"New balance: ${self.balance:,.2f}")
        else:
            print("Invalid deposit amount. Please enter a positive amount.")
        print("-" * 40)

    def withdraw(self, amount):
        """Method to withdraw money from the account"""
        if amount > 0:
            if amount <= self.balance:
```

```python
# You can also change it for a specific instance only
employee2.company = "Special Assignment Corp"
print("After changing employee2's company individually:")
employee1.display_details()
employee2.display_details()
employee3.display_details()
print()

class Rectangle:
    def __init__(self, length, width):
        """Constructor to initialize rectangle dimensions"""
        self.length = length
        self.width = width

    def calculate_area(self):
        """Calculate and return the area of the rectangle"""
        return self.length * self.width

    def calculate_perimeter(self):
        """Calculate and return the perimeter of the rectangle"""
        return 2 * (self.length + self.width)

    def display_details(self):
        """Display rectangle details including area and perimeter"""
        area = self.calculate_area()
        perimeter = self.calculate_perimeter()
        print(f"Rectangle Details:")
        print(f"Length: {self.length} units")
        print(f"Width: {self.width} units")
        print(f"Area: {area} square units")
        print(f"Perimeter: {perimeter} units")
        print("-" * 30)

print("Exercise 3: Using self")
print("=" * 30)

# Creating two rectangle objects
```



```python
class Employee:

    def display_details(self):
        """Method to display employee details"""
        print(f"Employee Details:")
        print(f"Name: {self.name}")
        print(f"Employee ID: {self.employee_id}")
        print(f"Salary: ${self.salary:,.2f}")
        print(f"Company: {self.company}")  # Accessing class variable
        print("-" * 30)

print("Exercise 2: Class and Instance Variables")
print("=" * 30)

# Creating three employee objects
employee1 = Employee("John Doe", "E001", 50000)
employee2 = Employee("Jane Smith", "E002", 60000)
employee3 = Employee("Mike Wilson", "E003", 55000)

# Displaying employee details
print("Initial Company Name:")
employee1.display_details()
employee2.display_details()
employee3.display_details()

# Changing the company name using class variable
print("After changing company name to 'XYZ Corporation':")
Employee.company = "XYZ Corporation"

# Observe the effect on all objects
employee1.display_details()
employee2.display_details()
employee3.display_details()

# You can also change it for a specific instance only
employee2.company = "Special Assignment Corp"
print("After changing employee2's company individually:")
```



```python
class Student:
    def __init__(self, name, roll_number, marks):
        """Constructor to initialize student attributes"""
        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def display_details(self):
        """Method to display student details"""
        print(f"Student Details:")
        print(f"Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Marks: {self.marks}")
        print("-" * 30)

# Creating two student objects
student1 = Student("Alice Johnson", "CS101", 85.5)
student2 = Student("Bob Smith", "CS102", 92.0)

# Displaying student details
print("Exercise 1: Student Class")
print("=" * 30)
student1.display_details()
student2.display_details()
print()

class Employee:
    # Class variable
    company = "ABC Technologies"

    def __init__(self, name, employee_id, salary):
        """Constructor to initialize instance variables"""
        self.name = name
        self.employee_id = employee_id
        self.salary = salary

    def display_details(self):
```